KnowOps AI: Phase 2.1 Architecture Report

Component: Gemini AI Core Integration (app/llm.py) & Vector Database Setup
Date: October 2026 | **Sprint:** sprint-1-(base) & Phase 2

1. Executive Summary & Purpose

In **Phase 2.1**, we built and established the central **AI Engine** of the KnowOps AI platform in app/llm.py. This module serves as the fundamental layer connecting the application to Google's Gemini Generative AI infrastructure and Neon PostgreSQL pgvector storage. Every subsequent feature—including knowledge base indexing, semantic search, grounded Q&A;, and autonomous agent loops—directly builds upon the functions created in this step.

> **Key Architectural Role:** Think of app/llm.py as the central *brain connector*. No other module creates direct, unmanaged HTTP requests or SDK bindings to Gemini. Instead, all embedding generation and generative chat calls pass through unified, tested, and resilient functions defined here.

2. Core Functions Implemented in app/llm.py

A. Client Initialization & Management — get_client()

Initializes and returns an authenticated instance of the official Google GenAI SDK client (google.genai.Client). Configured via app/config.py using the secure environment variable GEMINI_API_KEY.

```
def get_client() -> genai.Client:
    return genai.Client(api_key=settings.gemini_api_key)
```

B. Asymmetric Document Embedding — embed_document(text)

Converts knowledge base documents and text chunks into dense semantic vector representations. It uses model text-embedding-004 with task type **RETRIEVAL_DOCUMENT**. Crucially, it restricts output dimensionality to **768 dimensions** (via output_dimensionality=768), ensuring 100% compatibility with pgvector's HNSW index limits (which capped at 2,000 dimensions).

C. Query Embedding — embed_query(query)

Converts user questions into 768-dimensional vectors using task type **RETRIEVAL_QUERY**. Because user queries are typically short questions while stored chunks are long answers, Google's asymmetric embedding optimization maximizes cosine similarity accuracy between corresponding question-and-answer pairs.

D. Robust Chat Completion — chat_completion(...)

Interfaces with the Gemini chat models (gemini-flash-latest) to synthesize answers. Includes temperature control, system instructions (for role prompting and anti-hallucination constraints), and built-in exponential backoff retry mechanisms to handle transient 503 high-load spikes smoothly.

3. Vector Database Integration (PostgreSQL + pgvector)

Alongside the LLM layer, the Neon PostgreSQL database was aligned to store and index high-dimensional embeddings:

Database Component	Specification / Configuration	Purpose
Vector Column	chunks.embedding vector(768)	Stores 768-dimensional embeddings generated by embed_document.
Vector Index	HNSW index (vector_cosine_ops)	Provides sub-millisecond approximate nearest neighbor (ANN) cosine similarity search.
Metadata Storage	document_id, chunk_index, text, doc_metadata	Retains provenance, document headers, page numbers, and full text for citation.

4. Where and How Phase 2.1 is Used Across the Platform

Here is the exact lifecycle mapping showing how each function in app/llm.py powers the rest of KnowOps AI:

Phase / Feature	Target File	Function Used	Operational Purpose
Phase 2.2: Ingestion	scripts/ingest.py	embed_document()	Parses markdown/PDF/docs, splits into chunks, converts to 768d vectors, and writes to database.
Phase 2.3: RAG Search	app/rag.py	embed_query()	Takes user's question, embeds as vector, runs SQL cosine similarity: 1 - (embedding <=> query_vec).
Phase 2.4: Evaluation	tests/eval_rag.py	chat_completion()	Acts as LLM-as-a-judge to evaluate retrieval precision and factual accuracy against ground truth.
Phase 2.5: User API	app/main.py (/ask)	embed_query() chat_completion()	Public FastAPI endpoint: retrieves relevant chunks and generates a grounded response with source citations.
Phase 3: Agentic AI	app/agent.py	chat_completion()	Multi-turn reasoning, tool execution, and dynamic task delegation for IT operations incidents.

5. Verification and Test Results

A comprehensive test suite was executed against the live Gemini API and Neon database to validate implementation:

• **1. Client Authentication:** Successfully authenticated with Gemini API using secure key configuration.

• **2. Document Embeddings:** embed_document() produced exact 768-float vectors with normalized Euclidean norm.

• **3. Query Embeddings:** embed_query() correctly produced 768-float vectors optimized for retrieval tasks.

• **4. Semantic Cosine Matching:** Tested vector search logic. Related query (*'database failover'*) achieved high cosine similarity (>0.76) against disaster recovery chunks, whereas unrelated query (*'baking cake'*) yielded low similarity (<0.18).

• **5. Database Indexing:** Neon PostgreSQL verified with pgvector extension enabled and HNSW cosine index created successfully.

6. Next Steps: Roadmap to Phase 2.2

With the foundational AI engine complete and verified in Phase 2.1, the next step is **Phase 2.2 (Knowledge Base Ingestion)**. In Phase 2.2, we will implement scripts/ingest.py to load operations runbooks, system documentation, and incident guides, chunk the text with metadata preservation, call embed_document(), and populate the database for semantic search.